

**UNIVERSIDAD DE VIGO**

APS-GRIA – Curso 2025-2026

---

## **PRÁCTICA 4**

---

**IMU: Podómetro y Reconstrucción de Trayectoria**

*Guía Completa de Referencia Python*

Librerías: NumPy · Pandas · Matplotlib · SciPy  
Hardware: M5StickC Plus 2 (MPU-6886)

## Contenido

1. Introducción y Contexto
2. Librerías y Funciones Clave
3. Bloque 1 – Carga, Limpieza y Visualización
4. Bloque 2 – Detección de Pasos
5. Bloque 3 – Heading y Detección de Giros
6. Bloque 4 – Reconstrucción de Trayectoria PDR
7. Preguntas de Reflexión – Respuestas Orientativas
8. Referencia Rápida de Funciones
9. Errores Comunes y Soluciones

# 1. Introducción y Contexto

Esta práctica trabaja con datos reales capturados por la IMU (Inertial Measurement Unit) del M5StickC Plus 2, que integra el sensor MPU-6886 con acelerómetro y giróscopo de 3 ejes. Los datos fueron grabados mientras una persona caminaba, giraba y corría por los pasillos del edificio ESEI.

Todo el procesamiento se realiza offline en Python sobre Windows. No se programa el M5: se trabaja directamente con el fichero CSV exportado.

## 1.1 Formato de los datos

El fichero imu\_log2.csv contiene 14.101 muestras a ~172 Hz durante ~82 segundos. **Aquí van vuestros ficheros Este fichero es el de mis prueba.**

| Columna    | Unidad | Descripción                                                        |
|------------|--------|--------------------------------------------------------------------|
| ts_ms      | ms     | Timestamp monótono del ESP32                                       |
| ax, ay, az | g      | Aceleración en cada eje ( $1\text{ g} \approx 9.81\text{ m/s}^2$ ) |
| gx, gy, gz | °/s    | Velocidad angular. gz = yaw (plano horizontal)                     |

### Duplicados

El M5 lee la IMU más rápido de lo que el sensor actualiza su registro, generando filas con valores idénticos en timestamps consecutivos. Deben eliminarse en el Bloque 1.

## 1.2 Objetivo del pipeline

- Bloque 1: Cargar CSV, limpiar duplicados, calcular  $|a|$  y visualizar señales.
- Bloque 2: Detectar pasos como picos en  $|a|$  filtrada (Butterworth + find\_peaks).
- Bloque 3: Integrar gz para obtener heading, detectar giros significativos.
- Bloque 4: Reconstruir trayectoria 2D con PDR (paso fijo y variable Weinberg).

## 2. Librerías y Funciones Clave

Instalación de dependencias:

```
pip install numpy pandas matplotlib scipy
```

Imports necesarios en el script:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt, find_peaks
```

### 2.1 NumPy – Funciones utilizadas

| Función              | Uso en la práctica             | Ejemplo                           |
|----------------------|--------------------------------|-----------------------------------|
| np.sqrt(x)           | Calcular magnitud  a           | np.sqrt(ax**2 + ay**2 + az**2)    |
| np.diff(a)           | Calcular dt entre muestras     | dt = np.diff(t_s, prepend=t_s[0]) |
| np.cumsum(a)         | Integración numérica (Euler)   | heading = np.cumsum(gz * dt)      |
| np.deg2rad(a)        | Convertir heading a radianes   | h_rad = np.deg2rad(heading)       |
| np.cos / np.sin      | Componentes X/Y del paso       | dx = d * np.cos(h_rad)            |
| np.zeros(n)          | Inicializar arrays de posición | x = np.zeros(n_pasos + 1)         |
| np.where(cond, a, b) | Longitud condicional de paso   | d = np.where(amp>1, ..., 0)       |
| np.abs(a)            | Valor absoluto de gz           | en_giro = np.abs(gz) > umbral     |
| np.argmin(a)         | Buscar índice más cercano      | idx = np.argmin(np.abs(t - t0))   |

### 2.2 Pandas – Funciones utilizadas

| Función              | Uso                                               | Ejemplo                                     |
|----------------------|---------------------------------------------------|---------------------------------------------|
| pd.read_csv(f)       | Cargar datos IMU                                  | df = pd.read_csv('imu_log2.csv')            |
| df.drop_duplicates() | Eliminar duplicados globales (OJO: genera jitter) | df.drop_duplicates(subset=cols)             |
| df[cols].shift()     | Comparar filas consecutivas (diagnóstico)         | (df[cols] != df[cols].shift()).any(axis=1)  |
| np.interp()          | Remuestreo uniforme (corrige jitter)              | np.interp(t_uniform, t_orig, signal)        |
| df.reset_index()     | Reindexar tras filtrar                            | df = df[mask].reset_index(drop=True)        |
| df.iloc[:n]          | Seleccionar primeras n filas                      | df['gz'].iloc[:n_bias].mean()               |
| df.values            | Convertir a array NumPy                           | mag_filt = filtfilt(b, a, df['mag'].values) |

### 2.3 SciPy.signal – Funciones clave

#### 2.3.1 butter() – Diseño del filtro Butterworth

Diseña los coeficientes de un filtro Butterworth digital. El filtro Butterworth tiene una respuesta en frecuencia máximamente plana en la banda de paso.

```
from scipy.signal import butter

# Sintaxis: butter(N, Wn, btype, fs)
# N = orden del filtro (4 recomendado)
```

```
# Wn = frecuencia de corte en Hz (si se pasa fs)
# btype = 'low', 'high', 'bandpass', 'bandstop'
# fs = frecuencia de muestreo en Hz

# Ejemplo: filtro paso bajo, orden 4, corte 5 Hz, fs=172 Hz
b, a = butter(4, 5.0, btype='low', fs=172.0)

# b = coeficientes del numerador (feedforward)
# a = coeficientes del denominador (feedback)
```

### Frecuencia normalizada vs. fs

Si se pasa el parámetro fs, la frecuencia de corte Wn se expresa directamente en Hz. Sin fs, Wn debe normalizarse a [0, 1] donde  $1 = fs/2$  (Nyquist).

## 2.3.2 filtfilt() – Filtrado sin desfase

Aplica el filtro en ambas direcciones (forward-backward) para eliminar el desfase que introduce un filtro causal. Esto es posible porque trabajamos offline con datos completos.

```
from scipy.signal import filtfilt

# Sintaxis: filtfilt(b, a, x)
# b, a = coeficientes del filtro (de butter())
# x = señal a filtrar (array 1D)

# Ejemplo: filtrar la magnitud de aceleración
b, a = butter(4, 5.0, btype='low', fs=fs)
mag_filtrada = filtfilt(b, a, df['mag'].values)
```

### filtfilt vs lfilter

lfilter() aplica el filtro solo en una dirección e introduce desfase. Para análisis offline siempre usar filtfilt() que produce zero-phase filtering.

## 2.3.3 find\_peaks() – Detección de picos

Encuentra picos (máximos locales) en una señal 1D. Permite filtrar por altura, distancia mínima y prominencia.

```
from scipy.signal import find_peaks

# Sintaxis: find_peaks(x, height, distance, prominence, ...)
# x = señal donde buscar picos
# height = altura mínima del pico
# distance = distancia mínima entre picos (en MUESTRAS)
# prominence = prominencia mínima (altura relativa al entorno)

# Ejemplo: detectar pasos en |a| filtrada
picos, props = find_peaks(
    mag_filtrada,
    height=1.05,          # pico > 1.05 g
    distance=int(fs * 0.4), # mín 400 ms entre pasos
    prominence=0.05       # prominencia mínima
)

# picos = array de índices donde hay picos
# props = dict con 'peak_heights', 'prominences', etc.
# props['peak_heights'] -> altura de cada pico
# props['prominences'] -> prominencia de cada pico
```

| Parámetro | Tipo  | Efecto al aumentar               | Valor típico |
|-----------|-------|----------------------------------|--------------|
| height    | float | Menos picos (solo los más altos) | 1.02 – 1.20  |

| Parámetro  | Tipo           | Efecto al aumentar                        | Valor típico |
|------------|----------------|-------------------------------------------|--------------|
| distance   | int (muestras) | Elimina picos muy cercanos (doble conteo) | fs * 0.4     |
| prominence | float          | Ignora picos poco definidos               | 0.03 – 0.10  |

## 2.4 Matplotlib – Funciones de visualización

| Función                 | Uso                            | Ejemplo                                     |
|-------------------------|--------------------------------|---------------------------------------------|
| plt.subplots(n,m)       | Crear figura con subplots      | fig, axes = plt.subplots(3, 1, sharex=True) |
| ax.plot(x, y)           | Graficar señal                 | ax.plot(t, mag, linewidth=0.5)              |
| ax.axhline(y=v)         | Línea horizontal de referencia | ax.axhline(y=1.0, color='gray', ls='--')    |
| ax.axvspan(x0,x1)       | Sombrear región vertical       | ax.axvspan(t_ini, t_fin, alpha=0.15)        |
| ax.annotate()           | Añadir texto con flecha        | ax.annotate('+90°', xy=(t,h))               |
| ax.set_aspect('equal')  | Ejes iguales (trayectoria)     | ax.set_aspect('equal')                      |
| plt.cm.ScalarMappable() | Barra de color                 | sm = plt.cm.ScalarMappable(cmap=cmap)       |
| plt.colorbar(sm)        | Añadir colorbar                | cbar = plt.colorbar(sm, ax=axes)            |
| plt.tight_layout()      | Ajustar márgenes               | plt.tight_layout()                          |
| plt.savefig(f)          | Guardar imagen                 | plt.savefig('fig.png', dpi=150)             |

### 3. Bloque 1 – Carga, Limpieza y Visualización

Tiempo estimado: 25 minutos.

#### 3.1 Tarea 1 – Carga y tratamiento de duplicados

##### 3.1.1 El problema: lectura más rápida que el sensor

El M5StickC Plus 2 ejecuta su bucle de lectura a mayor velocidad de la que el MPU-6886 actualiza su registro interno. Esto produce muestras con valores de sensor idénticos a la fila anterior pero con timestamps distintos. El enunciado sugiere eliminar estos duplicados, pero hacerlo de forma ingenua introduce un problema serio: jitter temporal.

##### 3.1.2 Por qué eliminar duplicados genera jitter

Supongamos que el sensor actualiza cada ~5.8 ms (172 Hz) pero el ESP32 lee cada ~4 ms. La secuencia real podría ser:

| Fila | ts_ms | ax    | ¿Nuevo dato?   | dt si se elimina        |
|------|-------|-------|----------------|-------------------------|
| 0    | 1000  | 0.102 | Sí             | —                       |
| 1    | 1004  | 0.102 | No (duplicado) | (eliminada)             |
| 2    | 1008  | 0.115 | Sí             | 8 ms ← hueco            |
| 3    | 1012  | 0.115 | No (duplicado) | (eliminada)             |
| 4    | 1016  | 0.128 | Sí             | 8 ms ← hueco            |
| 5    | 1020  | 0.128 | No (duplicado) | (eliminada)             |
| 6    | 1024  | 0.128 | No (duplicado) | (eliminada)             |
| 7    | 1028  | 0.140 | Sí             | 12 ms ← hueco aún mayor |

Tras eliminar duplicados los intervalos dt pasan de ser uniformes (~4 ms) a irregulares (8, 8, 12 ms...). Esto es jitter temporal, y tiene consecuencias directas:

- Los filtros Butterworth asumen muestreo uniforme. Con jitter, la frecuencia de corte real varía muestra a muestra, distorsionando la respuesta en frecuencia.
- La integración Euler del heading (Bloque 3) acumula error proporcional a la irregularidad de dt.
- `find_peaks()` usa el parámetro `distance` en número de muestras. Con muestreo irregular, la misma distancia en muestras corresponde a intervalos de tiempo distintos.
- La estimación de  $f_s = \text{len(df)}/\text{duración}$  da una frecuencia media, pero la instantánea varía significativamente.

##### 3.1.3 Cuándo SÍ es aceptable eliminar duplicados

Eliminar duplicados consecutivos es razonable cuando se cumplen TODAS estas condiciones:

- **La proporción de duplicados es baja (<15%).** Si solo hay un duplicado esporádico cada 10-20 muestras, el jitter resultante es despreciable frente a la frecuencia de las señales de interés (pasos a 1-3 Hz).
- **El procesamiento posterior tolera muestreo no uniforme.** Si se usa integración con dt variable (como en el Bloque 3 de esta práctica), el efecto se compensa parcialmente.
- **Solo se buscan eventos de baja frecuencia.** Para detección de pasos (~2 Hz) un jitter de pocos ms es irrelevante. Para análisis espectral (FFT) sería crítico.
- **Es un ejercicio didáctico.** En esta práctica, el enunciado lo pide explícitamente. En producción, se usarían las alternativas descritas abajo.

### 3.1.4 Cuándo NO eliminar duplicados (alternativas)

En sistemas reales de navegación inercial, las alternativas correctas son:

| Alternativa              | Descripción                                                                                                                                             | Caso de uso                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Mantener todos los datos | Usar todas las muestras tal cual, con su timestamp original. Los valores repetidos no añaden información pero tampoco hacen daño si fs se calcula bien. | Pipeline simple, detección de pasos             |
| Remuestreo uniforme      | Interpolarse a una fs fija (ej. 100 Hz) con <code>np.interp()</code> o <code>scipy.interpolate</code> . Genera un eje temporal perfectamente regular.   | FFT, filtrado preciso, fusión de sensores       |
| Decimación controlada    | Reducir fs de forma controlada con <code>scipy.signal.decimate()</code> que aplica anti-aliasing antes de submuestrear.                                 | Reducir carga computacional manteniendo calidad |
| Data-ready interrupt     | Configurar el MPU-6886 con DRDY interrupt para que el ESP32 solo lea cuando hay dato nuevo. Elimina el problema en origen.                              | Firmware de producción                          |

### 3.1.5 Cómo detectar si hay jitter y cuantificarlo

Antes de decidir qué hacer con los datos, hay que diagnosticar el problema:

```
# ---- DIAGNÓSTICO DE JITTER ----

df = pd.read_csv('imu_log2.csv')
n_original = len(df)

# 1. Calcular dt entre muestras consecutivas (en ms)
dt_ms = np.diff(df['ts_ms'].values)

# 2. Estadísticas del intervalo de muestreo
print(f'dt medio:      {dt_ms.mean():.2f} ms')
print(f'dt mediana:    {np.median(dt_ms):.2f} ms')
print(f'dt std:         {dt_ms.std():.2f} ms')
print(f'dt mín:         {dt_ms.min():.2f} ms')
print(f'dt máx:         {dt_ms.max():.2f} ms')

# 3. Coeficiente de variación del muestreo (jitter relativo)
cv = dt_ms.std() / dt_ms.mean() * 100
print(f'Coeficiente de variación: {cv:.1f}%')
print(f' < 5%: jitter despreciable')
print(f' 5-15%: jitter moderado, aceptable para detección de pasos')
print(f' > 15%: jitter severo, considerar remuestreo')

# 4. Histograma de intervalos (visual)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(dt_ms, bins=50, color='steelblue', edgecolor='white')
axes[0].set_xlabel('dt (ms)')
axes[0].set_ylabel('Frecuencia')
axes[0].set_title('Distribución de intervalos de muestreo')
axes[0].axvline(x=dt_ms.mean(), color='red', ls='--',
                label=f'media={dt_ms.mean():.1f} ms')
axes[0].legend()

# 5. dt a lo largo del tiempo (detecta ráfagas de jitter)
axes[1].plot(df['ts_ms'].values[1:] / 1000, dt_ms, 's',
             markersize=1, color='steelblue')
axes[1].set_xlabel('Tiempo (s)')
axes[1].set_ylabel('dt (ms)')
axes[1].set_title('Intervalo de muestreo vs tiempo')
axes[1].axhline(y=dt_ms.mean(), color='red', ls='--')
plt.tight_layout()
```



```
plt.show()
```

### 3.1.6 Cómo corregir el jitter: remuestreo uniforme

Si se detecta jitter significativo ( $CV > 10\text{-}15\%$ ), la solución correcta es remuestrear la señal a una frecuencia uniforme mediante interpolación:

```
# ---- REMUESTREO UNIFORME ----

# Frecuencia de muestreo objetivo (elegir <= fs original)
fs_target = 100.0 # Hz, valor conservador

# Eje temporal original (puede ser irregular)
t_orig = (df['ts_ms'].values - df['ts_ms'].values[0]) / 1000.0

# Eje temporal uniforme
t_uniform = np.arange(t_orig[0], t_orig[-1], 1.0 / fs_target)

# Interpolar cada canal
cols_sensor = ['ax', 'ay', 'az', 'gx', 'gy', 'gz']
datos_uniformes = {'t_s': t_uniform}
for col in cols_sensor:
    datos_uniformes[col] = np.interp(t_uniform, t_orig, df[col].values)

df_uniform = pd.DataFrame(datos_uniformes)
print(f'Muestras originales: {len(df)}')
print(f'Muestras remuestreadas: {len(df_uniform)}')
print(f'fs efectiva: {fs_target} Hz (exacta)')

# Verificar: dt ahora es perfectamente constante
dt_new = np.diff(t_uniform)
print(f'dt std tras remuestreo: {dt_new.std()*1000:.4f} ms (≈ 0)')
```

#### **np.interp() es interpolación lineal**

Para señales de alta frecuencia o análisis espectral, considerar `scipy.interpolate.CubicSpline` para una interpolación más suave. Para detección de pasos, la interpolación lineal es suficiente.

### 3.1.7 Implementación para la práctica (enfoque del enunciado)

El enunciado pide explícitamente eliminar duplicados con `drop_duplicates()`. Para cumplir con el requisito didáctico manteniendo conciencia del problema:

```
# ---- IMPLEMENTACIÓN DEL ENUNCIADO (con diagnóstico) ----

df = pd.read_csv('imu_log2.csv')
n_original = len(df)

cols_sensor = ['ax', 'ay', 'az', 'gx', 'gy', 'gz']

# Detectar duplicados consecutivos
mascara = (df[cols_sensor] != df[cols_sensor].shift()).any(axis=1)
n_duplicados = (~mascara).sum()
pct_duplicados = n_duplicados / n_original * 100

print(f'Muestras originales: {n_original}')
print(f'Duplicados consec.: {n_duplicados} ({pct_duplicados:.1f}%)')

# Diagnóstico de jitter ANTES de eliminar
dt_antes = np.diff(df['ts_ms'].values)
cv_antes = dt_antes.std() / dt_antes.mean() * 100

# Eliminar (como pide el enunciado)
df = df[mascara].reset_index(drop=True)
```

```
# Diagnóstico de jitter DESPUÉS de eliminar
dt_despues = np.diff(df['ts_ms'].values)
cv_despues = dt_despues.std() / dt_despues.mean() * 100

print(f'\nJitter ANTES: CV = {cv_antes:.1f}%')
print(f'Jitter DESPUÉS: CV = {cv_despues:.1f}%')
print(f'Δ jitter: {cv_despues - cv_antes:+.1f}%')

if cv_despues > 15:
    print('⚠ Jitter severo: considerar remuestreo uniforme')
elif cv_despues > 5:
    print('⚠ Jitter moderado: aceptable para detección de pasos')
else:
    print('✓ Jitter bajo: muestreo aceptable')

# Convertir timestamp a segundos relativos
df['t_s'] = (df['ts_ms'] - df['ts_ms'].iloc[0]) / 1000.0
duracion = df['t_s'].iloc[-1]
fs = len(df) / duracion
print(f'\nMuestras finales: {len(df)}')
print(f'Duración: {duracion:.1f} s')
print(f'fs media: {fs:.1f} Hz')
```

### Resumen de buenas prácticas

1) Siempre diagnosticar el jitter ANTES y DESPUÉS de cualquier manipulación temporal. 2) Para esta práctica didáctica, eliminar duplicados es aceptable porque la proporción es baja y solo se detectan eventos de baja frecuencia. 3) En producción, usar data-ready interrupts en firmware o remuestreo uniforme en postprocesado.

## 3.2 Tarea 2 – Magnitud de la aceleración

La magnitud del vector aceleración combina los tres ejes en un único valor escalar. En reposo vale ~1.0 g (gravedad). Al caminar, los impactos producen picos por encima de 1 g.

```
# Fórmula: |a| = sqrt(ax² + ay² + az²)
df['mag'] = np.sqrt(df['ax']**2 + df['ay']**2 + df['az']**2)

print(f'|a| media: {df["mag"].mean():.3f} g')
print(f'|a| máx: {df["mag"].max():.3f} g') # Picos de carrera
```

## 3.3 Tarea 3 – Gráficas exploratorias

Se generan 3 subplots verticales compartiendo el eje X de tiempo:

```
fig, axes = plt.subplots(3, 1, sharex=True, figsize=(14, 8))
fig.suptitle('Bloque 1 - Señales IMU crudas', fontweight='bold')

# Subplot 1: |a| con línea de referencia en 1 g
axes[0].plot(df['t_s'], df['mag'], linewidth=0.5, color='steelblue')
axes[0].axhline(y=1.0, color='gray', linestyle='--', label='1 g')
axes[0].set_ylabel('|a| (g)')
axes[0].legend()

# Subplot 2: velocidad angular eje Z
axes[1].plot(df['t_s'], df['gz'], linewidth=0.5, color='darkorange')
axes[1].axhline(y=0, color='gray', linestyle='--', linewidth=0.5)
axes[1].set_ylabel('gz (°/s)')

# Subplot 3: componentes individuales
for col, c in [('ax', 'red'), ('ay', 'green'), ('az', 'blue')]:
    axes[2].plot(df['t_s'], df[col], linewidth=0.4, label=col, color=c)
axes[2].set_ylabel('Aceleración (g)')
axes[2].set_xlabel('Tiempo (s)')
axes[2].legend()
```

```
plt.tight_layout()  
plt.savefig('bloque1_senales.png', dpi=150, bbox_inches='tight')  
plt.show()
```

**sharex=True**

Compartir eje X permite hacer zoom sincronizado en los 3 subplots al explorar los datos interactivamente.

## 4. Bloque 2 – Detección de Pasos

Tiempo estimado: 30 minutos. Cada paso al caminar genera un pico característico en  $|a|$ . La cadencia típica es 1.5–2.0 pasos/s caminando y 2.5–3.0 pasos/s corriendo.

### 4.1 Tarea 1 – Filtrado paso bajo Butterworth

El filtrado elimina vibraciones de alta frecuencia que no corresponden a pasos. Se usa un Butterworth de orden 4 con frecuencia de corte 5 Hz.

```
# Calcular frecuencia de muestreo real
fs = len(df) / df['t_s'].iloc[-1]

# Diseñar filtro Butterworth paso bajo
b, a = butter(4, 5.0, btype='low', fs=fs)

# Aplicar filtfilt (zero-phase, sin desfase)
mag_filt = filtfilt(b, a, df['mag'].values)
```

El orden 4 proporciona una caída de 80 dB/década en la banda de rechazo. Frecuencias de corte típicas para detección de pasos: 3–8 Hz. Valores demasiado bajos (<2 Hz) eliminan los propios picos de paso; valores demasiado altos (>15 Hz) dejan ruido que genera falsos positivos.

### 4.2 Tarea 2 – Detección de picos con find\_peaks

```
# Distancia mínima en muestras (400 ms)
dist_muestras = int(fs * 0.4)

picos, props = find_peaks(
    mag_filt,
    height=1.05,          # solo picos > 1.05 g
    distance=dist_muestras, # mín 400 ms entre pasos
    prominence=0.05       # prominencia mínima
)

n_pasos = len(picos)
cadencia = n_pasos / df['t_s'].iloc[-1]
print(f'Pasos: {n_pasos}, Cadencia: {cadencia:.2f} pasos/s')

# Acceder a las alturas de los picos:
alturas = props['peak_heights'] # array con la altura de cada pico
prominencias = props['prominences'] # array con la prominencia
```

La prominencia mide cuánto sobresale un pico respecto a los valles circundantes. Un pico con gran altura absoluta pero poca prominencia podría ser simplemente un punto en una meseta, no un verdadero paso.

### 4.3 Tarea 3 – Visualización de pasos detectados

```
fig, ax = plt.subplots(figsize=(14, 4))
ax.plot(df['t_s'], mag_filt, linewidth=0.6, color='steelblue',
        label='|a| filtrada')

# Marcar picos detectados con triángulos rojos
ax.plot(df['t_s'].values[picos], mag_filt[picos],
        'rv', markersize=4, label=f'Pasos ({n_pasos})')

# Línea de umbral
ax.axhline(y=1.05, color='red', linestyle=':', linewidth=0.7,
           label='Umbral = 1.05 g')
```

```
ax.set_xlabel('Tiempo (s)')
ax.set_ylabel('|a| (g)')
ax.set_title(f'Detección de pasos [{n_pasos} pasos, {cadencia:.2f} p/s]')
ax.legend(loc='upper right')
plt.tight_layout()
plt.savefig('bloque2_pasos.png', dpi=150)
plt.show()
```

## 4.4 Experimento C – Segmentación caminar vs correr

La amplitud de los picos permite distinguir automáticamente los modos de locomoción:

```
UMBRAL_CORRER = 1.4 # g
pasos_corriendo = np.sum(props['peak_heights'] > UMBRAL_CORRER)
pasos_andando = n_pasos - pasos_corriendo
print(f'{pasos_andando} andando, {pasos_corriendo} corriendo')
```

## 5. Bloque 3 – Heading y Detección de Giros

Tiempo estimado: 25 minutos. El giróscopo mide velocidad angular instantánea (°/s). Integrándola se obtiene el ángulo acumulado (heading).

### 5.1 Tarea 1 – Compensación de bias

El bias es un offset constante en la lectura del giróscopo. Sin compensarlo, el heading acumula un error lineal con el tiempo. El MPU-6886 tiene un bias típico de 1–3 °/s.

```
# Ventana inicial de 0.5 s (asumiendo reposo al inicio)
n_bias = int(fs * 0.5)
bias_gz = df['gz'].iloc[:n_bias].mean()
print(f'Bias gz: {bias_gz:.3f} °/s')

# Corregir toda la señal
gz_corr = df['gz'].values - bias_gz

# Sin compensar: tras 80 s con bias de 2 °/s -> 160° de error
# Con compensar: el drift residual se reduce a unos pocos grados
```

### 5.2 Tarea 2 – Filtrado e integración del heading

```
# Filtro paso bajo a 2 Hz para reducir ruido antes de integrar
b, a = butter(4, 2.0, btype='low', fs=fs)
gz_filt = filtfilt(b, a, gz_corr)

# Calcular dt entre muestras consecutivas
dt = np.diff(df['t_s'].values, prepend=df['t_s'].values[0])
dt[0] = 0 # primer intervalo es 0

# Integración Euler: heading(i) = heading(i-1) + gz(i) * dt(i)
heading = np.cumsum(gz_filt * dt)

print(f'Heading final: {heading[-1]:.1f}°')
```

#### Integración trapezoidal (alternativa más precisa)

Para mayor precisión se puede usar `np.trapz()` o integrar con la regla del trapecio: `heading(i) = heading(i-1) + 0.5*(gz(i) + gz(i-1)) * dt(i)`. La diferencia es pequeña con muestreo rápido (~172 Hz).

### 5.3 Tarea 3 – Detección de giros

Un giro significativo se detecta cuando `|gz_filtrado|` supera un umbral (20 °/s) durante más de 200 ms:

```
GIRO_UMBRAL = 20.0 # °/s
GIRO_DUR_MIN = 0.2 # segundos

en_giro = np.abs(gz_filt) > GIRO_UMBRAL
giros = []
inicio = None

for i in range(len(en_giro)):
    if en_giro[i] and inicio is None:
        inicio = i
    elif not en_giro[i] and inicio is not None:
        dur = df['t_s'].values[i] - df['t_s'].values[inicio]
        if dur >= GIRO_DUR_MIN:
            delta_h = heading[i] - heading[inicio]
            giros.append({
                'idx_inicio': inicio,
                'idx_fin': i,
                't_inicio': df['t_s'].values[inicio],
                't_fin': df['t_s'].values[i],
```

```

        'duracion': dur,
        'delta_heading': delta_h,
        'direccion': 'D' if delta_h > 0 else 'I'
    })
    inicio = None

print(f'Giros detectados: {len(giros)}')
for g in giros:
    print(f't={g["t_inicio"]:.1f}s, dur={g["duracion"]:.2f}s, '
          f'delta={g["delta_heading"]:.0f}°, dir={g["direccion"]}')

```

## 5.4 Tarea 4 – Visualización de heading y giros

```

fig, axes = plt.subplots(2, 1, sharex=True, figsize=(14, 6))

# Subplot 1: gz original vs filtrado
axes[0].plot(df['t_s'], gz_corr, lw=0.3, color='lightcoral',
             alpha=0.6, label='gz corregido')
axes[0].plot(df['t_s'], gz_filt, lw=0.8, color='red',
             label='gz filtrado')
axes[0].axhline(y=20, color='gray', ls=':', lw=0.7)
axes[0].axhline(y=-20, color='gray', ls=':', lw=0.7)
axes[0].set_ylabel('gz (°/s)')
axes[0].legend()

# Subplot 2: heading acumulado + zonas de giro
axes[1].plot(df['t_s'], heading, lw=1.0, color='navy')
for g in giros:
    axes[1].axvspan(g['t_inicio'], g['t_fin'],
                   alpha=0.15, color='orange')
    t_mid = (g['t_inicio'] + g['t_fin']) / 2
    idx = np.argmin(np.abs(df['t_s'].values - t_mid))
    axes[1].annotate(f'{g["delta_heading"]:.0f}°',
                    xy=(t_mid, heading[idx]), fontsize=7,
                    bbox=dict(boxstyle='round', fc='yellow', alpha=0.7))

axes[1].set_ylabel('Heading (°)')
axes[1].set_xlabel('Tiempo (s)')
plt.tight_layout()
plt.savefig('bloque3_heading.png', dpi=150)
plt.show()

```

## 6. Bloque 4 – Reconstrucción de Trayectoria PDR

Tiempo estimado: 40 minutos. Pedestrian Dead Reckoning (PDR) combina la detección de pasos con la orientación estimada para reconstruir la trayectoria en 2D.

### 6.1 Fórmulas fundamentales del PDR

En cada paso  $n$  se actualiza la posición:

$$\begin{aligned}x(n) &= x(n-1) + d(n) * \cos(\text{heading}(n)) \\ y(n) &= y(n-1) + d(n) * \sin(\text{heading}(n))\end{aligned}$$

Donde  $d(n)$  es la longitud del paso, calculada con dos modelos:

| Modelo        | Fórmula                              | Características                                 |
|---------------|--------------------------------------|-------------------------------------------------|
| A – Paso fijo | $d = 0.70 \text{ m}$                 | Simple, constante para todos los pasos          |
| B – Weinberg  | $d = K * (\text{pico} - 1.0)^{0.25}$ | Variable, proporcional a la intensidad del paso |

### 6.2 Tarea 1 – Heading en cada paso

```
# Obtener tiempo, heading y amplitud en cada pico
t_pasos = df['t_s'].values[picos]
h_pasos = heading[picos]          # heading en el instante del paso
amp_pasos = props['peak_heights'] # amplitud del pico de aceleración
```

### 6.3 Tarea 2 – Modelo A (paso fijo)

```
PASO_FIJO = 0.70 # metros

# Convertir heading a radianes
h_rad = np.deg2rad(h_pasos)

# Arrays de posición (n_pasos + 1 puntos, empezando en origen)
x_a = np.zeros(len(picos) + 1)
y_a = np.zeros(len(picos) + 1)

for i in range(len(picos)):
    x_a[i+1] = x_a[i] + PASO_FIJO * np.cos(h_rad[i])
    y_a[i+1] = y_a[i] + PASO_FIJO * np.sin(h_rad[i])

dist_total_a = len(picos) * PASO_FIJO
error_cierre_a = np.sqrt(x_a[-1]**2 + y_a[-1]**2)
```

### 6.4 Tarea 3 – Modelo B (paso variable Weinberg)

```
K = 0.60

# d(n) = K * (pico(n) - 1.0)^0.25
# Si pico < 1.0 (raro), d = 0
d_var = np.where(
    amp_pasos > 1.0,
    K * (amp_pasos - 1.0) ** 0.25,
    0.0
)

x_b = np.zeros(len(picos) + 1)
y_b = np.zeros(len(picos) + 1)

for i in range(len(picos)):
    x_b[i+1] = x_b[i] + d_var[i] * np.cos(h_rad[i])
```



```

y_b[i+1] = y_b[i] + d_var[i] * np.sin(h_rad[i])

dist_total_b = np.sum(d_var)
error_cierre_b = np.sqrt(x_b[-1]**2 + y_b[-1]**2)

```

### Raíz cuarta vs amplitud directa

La raíz cuarta comprime el rango dinámico. Ejemplo: pico 1.2 g (marcha) vs 1.8 g (carrera). Amplitud directa:  $0.8/0.2 = 4x$  más largo. Raíz cuarta:  $0.8^{0.25} / 0.2^{0.25} = 0.946/0.669 = 1.41x$ . La raíz cuarta es mucho más realista.

## 6.5 Tarea 4 – Visualización con codificación de color

```

# Calcular velocidad local (cadencia en ventana de 5 pasos)
ventana = 5
velocidad = np.zeros(len(picos))
for i in range(len(picos)):
    i0 = max(0, i - ventana // 2)
    i1 = min(len(picos)-1, i + ventana // 2)
    if i1 > i0:
        dt_local = t_pasos[i1] - t_pasos[i0]
        velocidad[i] = (i1 - i0) / dt_local if dt_local > 0 else 0

fig, axes = plt.subplots(1, 2, figsize=(16, 7))
cmap = plt.cm.RdYlGn_r # rojo=rápido, verde=lento
v_min, v_max = velocidad.min(), velocidad.max()

for ax, x_arr, y_arr, titulo in [
    (axes[0], x_a, y_a, 'Modelo A - Paso fijo'),
    (axes[1], x_b, y_b, 'Modelo B - Variable')
]:
    for i in range(len(picos)):
        c = cmap((velocidad[i]-v_min) / (v_max-v_min+1e-9))
        ax.plot(x_arr[i:i+2], y_arr[i:i+2], color=c, lw=1.5)

    ax.plot(x_arr[0], y_arr[0], 'go', ms=10, label='Inicio')
    ax.plot(x_arr[-1], y_arr[-1], 'rs', ms=10, label='Final')
    ax.plot([x_arr[-1], x_arr[0]], [y_arr[-1], y_arr[0]],
            'k--', lw=0.8, alpha=0.5)
    ax.set_aspect('equal')
    ax.set_title(titulo)
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)

# Barra de color
sm = plt.cm.ScalarMappable(cmap=cmap,
                           norm=plt.Normalize(v_min, v_max))
sm.set_array([])
cbar = plt.colorbar(sm, ax=axes, shrink=0.6)
cbar.set_label('Cadencia local (pasos/s)')

plt.tight_layout()
plt.savefig('bloque4_trayectoria.png', dpi=150)
plt.show()

```

## 7. Preguntas de Reflexión – Respuestas Orientativas

### R1 – Drift del giróscopo y filtro de Kalman

Si el bias medido es de  $\sim 2^\circ/\text{s}$  y el recorrido dura 10 min (600 s), el heading falso acumulado sería  $2 \times 600 = 1200^\circ$  (más de 3 vueltas completas). Esto hace el sistema inviable sin corrección.

El filtro complementario combina datos del giróscopo (fiable a corto plazo) con el acelerómetro o magnetómetro (fiable a largo plazo) mediante una ponderación en frecuencia:  $\text{heading} = \alpha \cdot (\text{heading\_gyro}) + (1-\alpha) \cdot (\text{heading\_mag})$ . Típicamente  $\alpha \approx 0.96\text{--}0.98$ .

El filtro de Kalman es más sofisticado: modela explícitamente la incertidumbre de cada sensor y calcula la combinación óptima en cada paso temporal, minimizando el error cuadrático medio.

### R2 – Raíz cuarta vs amplitud directa

Comparación numérica para  $\text{pico\_marcha} = 1.2 \text{ g}$  y  $\text{pico\_carrera} = 1.8 \text{ g}$ :

| Método           | $\Delta a$ marcha    | $\Delta a$ carrera   | Ratio carrera/marcha |
|------------------|----------------------|----------------------|----------------------|
| Amplitud directa | 0.2                  | 0.8                  | 4.0x                 |
| Raíz cuarta      | $0.2^{0.25} = 0.669$ | $0.8^{0.25} = 0.946$ | 1.41x                |

La amplitud directa haría que un paso de carrera fuese 4 veces más largo que uno de marcha, lo cual es irreal (la diferencia real es  $\sim 1.3\text{--}1.5x$ ). La raíz cuarta comprime este rango a valores mucho más realistas.

### R3 – Corrección con puntos de referencia

El error PDR crece como  $\sqrt{N}$  (random walk). Con puntos de referencia conocidos (puertas, esquinas) se puede aplicar map matching: al detectar un giro de  $\sim 90^\circ$  que coincida con una esquina conocida del plano, se fuerza la posición estimada a la posición real de esa esquina, reseteando el error acumulado. También se pueden usar técnicas de particle filter o graph-based SLAM.

### R4 – Dispositivo con orientación libre

Si el M5 rota libremente en el bolsillo, la detección de pasos sigue funcionando ya que la magnitud  $|a|$  es invariante a la rotación (es la norma del vector, independiente de la orientación del dispositivo). Sin embargo, la detección de heading falla porque  $g_z$  ya no corresponde necesariamente al giro en el plano horizontal. Se necesitaría un algoritmo de estimación de orientación (AHRS) para separar las componentes de rotación del dispositivo de las del usuario.

### R5 – IMU vs WiFi vs BLE vs UWB

| Sistema          | Precisión      | Infraestructura       | Mejor escenario              |
|------------------|----------------|-----------------------|------------------------------|
| IMU (PDR)        | 1–5% distancia | Ninguna               | Recorridos cortos, sin infra |
| WiFi fingerprint | 2–5 m          | APs existentes + mapa | Edificios con WiFi denso     |
| BLE beacons      | 1–3 m          | Balizas cada 5–10 m   | Retail, museos, hospitales   |
| UWB              | 10–30 cm       | Anclas UWB dedicadas  | Almacenes, logística, robots |

La IMU es la mejor opción cuando no hay infraestructura disponible, para recorridos cortos (<2 min), o como complemento de otros sistemas (sensor fusion).

## **R6 – Detección de escaleras**

En reposo horizontal,  $a_z \approx 1.0$  g y  $a_x, a_y \approx 0$ . Al subir escaleras con el dispositivo en la mano, la componente vertical ( $a_z$  si el dispositivo está horizontal) muestra un patrón característico de ascenso: la media de  $a_z$  disminuye ligeramente ( $\approx 0.95$ – $0.98$  g) durante la subida porque parte de la gravedad se “transfiere” a la componente de aceleración lineal vertical. También se puede integrar la aceleración vertical para estimar el desplazamiento en Z.

## 8. Referencia Rápida de Funciones

Resumen de todas las llamadas a funciones utilizadas en la práctica, organizadas por bloque:

| Bloque | Función                                | Propósito                          |
|--------|----------------------------------------|------------------------------------|
| B1     | pd.read_csv()                          | Cargar CSV                         |
| B1     | df[cols].shift()                       | Detectar duplicados consecutivos   |
| B1     | np.sqrt()                              | Magnitud $ a $                     |
| B1     | plt.subplots(3,1,sharex=True)          | 3 subplots sincronizados           |
| B2     | butter(4, 5.0, 'low', fs=fs)           | Coeficientes Butterworth           |
| B2     | filtfilt(b, a, signal)                 | Filtrado zero-phase                |
| B2     | find_peaks(signal, height, dist, prom) | Detección de pasos                 |
| B3     | df['gz'].iloc[:n].mean()               | Estimación de bias                 |
| B3     | np.diff() + np.cumsum()                | Integración Euler del heading      |
| B3     | np.abs(gz) > umbral                    | Máscara de giros activos           |
| B4     | heading[picos]                         | Heading en instante de cada paso   |
| B4     | np.deg2rad()                           | Conversión a radianes para cos/sin |
| B4     | np.cos/sin * d                         | Componentes X/Y del desplazamiento |
| B4     | np.where(cond, val, 0)                 | Longitud de paso condicional       |
| B4     | plt.cm.RdYlGn_r                        | Colormap velocidad (rojo=rápido)   |

## 9. Errores Comunes y Soluciones

| Error                           | Causa                                     | Solución                                          |
|---------------------------------|-------------------------------------------|---------------------------------------------------|
| find_peaks no detecta picos     | height demasiado alto o señal sin filtrar | Bajar height a 1.02, verificar filtfilt aplicado  |
| Heading crece sin parar         | No se compensó el bias de gz              | Restar media de gz en reposo (primeros 0.5 s)     |
| Trayectoria es una espiral      | Heading en grados pasado a cos/sin        | Usar np.deg2rad() antes de cos/sin                |
| distance de find_peaks da error | Valor float en vez de int                 | Usar int(fs * 0.4) con int()                      |
| filtfilt falla: padlen          | Señal demasiado corta para el orden       | Reducir orden del filtro o verificar datos        |
| Giros no detectados             | Umbral gz muy alto                        | Probar con 10–15 °/s en vez de 20                 |
| Trayectoria no cierra           | Normal: es drift acumulado del PDR        | Medir error de cierre, no esperar cierre perfecto |
| Picos dobles (doble conteo)     | distance muy baja                         | Aumentar a fs*0.5 (500 ms entre pasos)            |

### Consejo general

Ejecuta los bloques uno a uno, verificando visualmente cada gráfica antes de pasar al siguiente. Los errores en un bloque se propagan a todos los siguientes, especialmente el heading del Bloque 3 que afecta directamente a la trayectoria del Bloque 4.

*Documento generado como material de apoyo para la Práctica 4 de APS-GRIA.*

*Universidade de Vigo – Curso 2025-2026*